

基于通信微内核的 AI 自设计/自演进 RAN 原型系统思考-ouyang

1.总体思路

定位为面向未来 AI Native RAN 的“基于通信微内核的 AI 自设计/自演进 RAN 原型系统”，是对当前 AI-RAN 范式的一次纵深推进，通信系统不再是“一次性人工设计的固化系统”，而是以微内核为可信底座、由 AI 在受限空间内动态生成与演进的系统。

传统 AI-RAN 大多是在 4G/5G 既有 RAN 架构内引入 AI 能力，例如用 AI 增强信道估计、波束管理、资源调度、流量预测和节能控制，本质上仍属于“人工设计通信系统+AI 局部优化”的范式。而该思路尝试进一步推进到“AI 按需生成通信能力”的新阶段，更强调 AI 生成而非 AI 加速：系统初始不预置完整、固定的空口和协议栈，而是保留一个可信、稳定、低复杂度、可回退的最小通信内核，在此基础上由 AI 根据任务需求、信道状态、频谱环境、终端能力、算力资源和 QoS 约束，动态选择、组合、配置甚至受限生成通信功能模块，包括波形、调制编码、导频结构、信道估计、接收机、接入控制、资源调度、路由与跨层优化策略等。其本质是推动通信系统从“人工预设方案”向“AI 自描述、自生成、自验证、自部署”的方向演进，可视为 AI self-designing / self-evolving RAN 的早期探索形态，尤其适合卫星通信、应急通信、专用通信、无人集群等环境变化剧烈、任务差异大、传统固定体制难以完全覆盖的场景。

第一阶段建议构建一个小规模最小闭环原型，即“通信微内核 + AI 模块编排 + 数字孪生验证 + SDR/仿真演示”。初期优先验证六个问题：

- 1) 最小通信内核是否可行
- 2) 通信模块是否能标准化插件化
- 3) AI 是否能根据任务和环境生成候选方案
- 4) 候选方案是否能在数字孪生中自动验证
- 5) 验证通过的模块是否能动态部署
- 6) 异常情况下是否能安全回退。

总体而言，通过可控原型证明“AI 按需生成通信能力”的可行性，为未来 AI 原生、自设计、自演进 RAN 系统奠定理论和工程基础。

最小通信内核是整个系统的可信底座，不追求高吞吐和复杂业务承载，而强调“活着、可控、可恢复”，借鉴 Linux Kernel 的 stable ABI 思想：主版本号不变即向后兼容，接口不可频繁改动。建议其只保留最必要能力：固定低速可靠

控制信道，可采用保守的 BPSK/QPSK、低码率编码、高导频密度和简单 ARQ 机制；基础同步与节点发现，用于建立最初通信关系；最小信令协议，用于传输状态上报、任务描述、模块加载命令、回退指令和 ACK/NACK；状态监测接口，用于采集 SNR、SINR、BLER、吞吐、时延、干扰、频偏、多普勒、队列长度、终端电量和算力占用等信息；模块加载与卸载机制，用于将验证通过的通信模块动态部署到数据面；以及安全回退机制，一旦新模块异常，系统立即卸载异常模块并切回最小控制信道。这个内核类似 Linux 微内核思想，核心功能越少越好，但接口必须稳定、可信、可扩展。

协议栈模块化是该方向能否落地的关键。虽然 4G/5G 已经具备分层协议栈结构，但要支持 AI 自生成和自演进，还需要进一步将功能插件化。例如，PHY 层可拆解为调制、编码、导频、信道估计、同步增强、均衡检测、神经接收机等模块；MAC 层可拆解为 MCS 选择、HARQ、调度器、随机接入、功率控制和资源分配模块；RLC/PDCP/RRC 层可提供分段重组、缓存管理、QoS 映射、安全加密、能力协商和连接配置模块；跨层策略可封装为低时延、高可靠、抗干扰、高吞吐、低功耗等模式。每个模块必须带有统一描述文件，包括模块名称、类型、输入输出接口、参数范围、资源需求、适用场景、风险等级、验证状态、版本号、签名信息和回退策略，形成类似操作系统驱动模型和 ABI 规范的通信模块生态。

AI 生成机制建议采用“受约束生成”，而非完全自由生成。第一阶段重点做参数生成，如 MCS 等级、编码率、导频密度、发射功率、HARQ 次数、调度权重和接入退避参数；第二阶段做模块组合，如在低 SNR 场景选择低阶调制、低码率编码、高导频密度和保守 HARQ，在高 SNR 场景选择高阶调制、高码率编码和低导频开销；第三阶段再探索受限策略代码生成，例如 MCS 映射函数、调度权重函数和干扰规避规则。这里可以借鉴 Linux eBPF 思想，设计“通信 eBPF”机制，使 AI 只能生成小型、可验证、有限时执行、不能越界、不能死循环的策略插件，从而降低 AI 生成内容的安全风险。

所有候选方案都必须先进入数字孪生沙箱，完成接口一致性检查、语法检查、协议状态机检查、链路级仿真、简化系统级仿真、资源占用评估、实时性评估、安全约束检查和回退测试。只有满足可靠性、时延、吞吐、功率、频谱、算力和回退要求的方案，才能被签名准入并部署到仿真平台、GNU Radio/USRP 等 SDR 平台，或进一步扩展到 srsRAN、OpenAirInterface 等开源 4G/5G 协议栈中。整个闭环应遵循“生成—验证—签名—部署—监控—回退”的原则。

2. 架构设计

2.1 架构核心业务流程：

场景/任务输入 → AI 多智能体编排 → 模块库智能选配 → 生成候选方案
→ 数字孪生全维度验证评分 → 签名准入下发 → 运行时动态加载重构 → 实时监控异常回退 → 数据回流持续迭代。

2.2 架构核心设计思路：

✓ **内核极简中心化：**核心内核只做调度与接口定义，业务能力全部下沉分层插件化。

✓ **安全内生全链路：**从可信启动、策略约束、模块签名、eBPF 策略校验全流程安全闭环。

✓ **控制面保底设计：**独立低速可靠控制信道作为生命线，故障可强制降级回退。

✓ **全模块化可加载：**通信功能标准化 ABI 接口，支持热加载、版本切换、原子回滚。

✓ **软硬件完全解耦：**HAL 屏蔽硬件差异，同一协议栈可跨 SDR/FPGA/SoC 部署。

✓ **监控自愈闭环：**实时监控—异常判决—自动回退—AI 重优化，形成自治闭环。

✓ **开放外部生态：**原生支持 AI 编排、数字孪生仿真、多行业上层业务无缝接入。

2.3 架构包括以下模块：

(1) 任务场景输入层

面向卫星、应急、专用、无人系统、6G/NTN 等业务场景；输入任务意图、环境状态、终端平台能力，作为整个智能通信架构的原始输入。

(2) AI 通信编排层

以 AI 通信编排器为核心，内置任务理解、环境分析、模块选择、参数生成、受限 eBPF 代码生成多智能体；

搭配安全规则引擎做功率、频谱、时延、算力硬约束过滤，输出合规优化策略。

(3) 通信模块仓库层

统一管理 PHY、MAC、RLC/PDCP/RRC、跨层策略四大类可插拔通信模块；附带模块元数据、版本、ABI 接口、签名、回退策略，支撑 AI 智能选取。

(4) 候选通信方案生成层

基于 AI 编排+模块库组合，生成低 SNR 可靠、高吞吐、抗干扰等典型可落地通信配置方案。

(5) 数字孪生验证沙箱层

对候选方案做链路级仿真、系统级仿真、协议一致性、资源实时性、安全合规、故障回退全维度测试；

综合评分后签名准入，才可进入实际运行时加载。

(6) 通信运行时平台层

核心包含：最小通信内核、硬件抽象 HAL、模块动态加载器、可重构协议栈、运行监控器；

支持模块热加载、协议栈可重构、异常自动触发内核安全回退。

(7) 实验与部署平台层

覆盖仿真平台、SDR 硬件平台、开源 4G/5G 协议栈、O-RAN/AI-RAN 四大落地载体，支撑从仿真到实网试点全流程验证。

(8) 运行反馈持续演进闭环

实测数据回流 → 数字孪生模型校准 → 模块性能迭代 → AI 策略再生成 → 模块库版本更新，形成自学习、自优化、自演进闭环架构。

3.最小通信内核

最小通信内核以可信、稳定、低复杂度、可回退、可扩展为核心目标，遵循“内核尽量精简、接口绝对稳定”设计原则。

整体采用内核中枢 + 分层解耦 + 模块化可加载 + 安全可信 + 监控自愈回退 + 外部智能编排架构，划分为功能分层，各层职责独立、接口标准化、可热加载、异常可自动回退，支持 AI 编排、数字孪生仿真、多类上层通信业务接入。

(1) 核心内核层（总调度中枢）

核心节点：最小通信内核

定位：整个系统总入口、总调度、总管控中枢

核心职责

- 统筹调度所有安全、控制、感知、模块、运行时、监控等各分层能力
- 定义全局稳定接口，维持内核最小化，不堆砌业务逻辑
- 统一管控模块加载、策略下发、异常回退、资源调度全流程

(2) 安全可信根与权限控制层

包含两大能力单元：可信启动与身份认证、安全策略与权限控制

核心能力

1. 可信启动与身份认证

- 节点身份可信认证、通信模块签名校验
- 全局可信根与密钥全生命周期管理
- 拦截非法未签名模块加载，筑牢启动安全基线

2. 安全策略与权限控制

- 功率、频谱、时延、算力硬边界约束
- 模块访问权限管控、业务黑白名单机制
- 全局资源预算隔离，防止越权与资源滥用

依赖关系：由最小通信内核驱动，向下约束模块加载、策略插件校验。

(3) 低速可靠控制面层

包含：低速可靠控制信道、基础同步与发现、最小信令协议

定位：系统生命线安全通道，极端场景保底通信与信令承载

核心能力

1. 低速可靠控制信道

- 固定波形/固定帧结构、BPSK/QPSK 低阶调制
- 低码率 FEC、高导频密度、简易 ARQ 重传
- 作为异常回退后的保底安全通信通道

2. 基础同步与发现

- 时间/频率同步、小区与邻节点发现
- 基础广播信道、初始接入信标承载

3. 最小信令协议

- 任务/状态上报、模块加载/卸载/回退命令
- 信令 ACK/NACK 可靠确认，极简信令不冗余

依赖：内核统一管控，同步发现支撑控制信道，控制信道承载最小信令。

(4) 全网状态感知层

包含：状态采集器、任务与能力描述接口

定位：环境+终端+业务全维度感知，为智能调度、模块适配提供输入

核心能力

1. 状态采集器

- 无线指标：SNR/SINR、BER/BLER、RSSI/RSRP/RSRQ
- 信道特性：频偏时偏、多径、多普勒、干扰强度

- 系统业务：队列长度、业务速率、终端电量等全量采集

2. 任务与能力描述接口

- 业务 QoS、可靠性、时延、吞吐需求建模
- 终端能力、平台算力、可用带宽、发射功率约束抽象

依赖：内核驱动，感知数据汇聚后映射为业务与能力描述，对外提供 AI、上层业务接口。

(5) 标准化可加载模块层

包含：通信模块 ABI/API、模块描述文件、模块加载/卸载器

定位：通信功能插件化、标准化、可热插拔、版本可灰度切换

核心能力

1. 通信模块 ABI/API：统一输入输出格式、参数范围、调用时序、状态机、错误码、回退标准接口

2. 模块描述文件：定义模块类型版本、资源需求、适用场景、风险等级、签名与回退策略

3. 模块加载卸载器：签名校验、ABI 兼容性检查、资源预留、冷热加载、版本原子切换与回滚

约束：受安全可信层策略校验，由内核统一管控生命周期。

(6) 通信 eBPF 受限策略插件层

包含：受限策略插件机制、策略验证器

设计思想：借鉴 Linux eBPF，实现通信层安全可控的用户自定义策略

核心能力

1. 受限策略插件：支持 MCS 映射、调度权重、接入控制、干扰规避等自定义规则

- 强约束：禁止死循环、内存越界、无限制执行时长

2. 策略验证器：语法/接口/循环边界/内存访问/执行时长/安全规则全维度校验

依赖：受安全策略层约束，校验通过后注入模块加载器生效。

(7) 硬件抽象 HAL 层

核心单元：通信硬件抽象层 HAL

定位：屏蔽底层硬件差异，实现软硬件解耦

覆盖硬件：RF 前端、SDR/USRP、FPGA/SoC、CPU/GPU 加速器

能力抽象：采样率、带宽、发射功率、硬件加速能力统一封装

价值：上层协议与模块无需感知具体硬件平台，可跨硬件平滑迁移。

(8) 实时运行时与数据面层

包含：实时运行调度器、数据面可加载模块区

核心能力

1. 实时运行调度器

- 帧级高精度调度、模块执行时限隔离
- CPU/GPU/FPGA 多资源统一分配
- 控制面优先级最高，数据面支持动态可重构

2. 数据面可加载模块区

- 调制、编码、信道估计、接收机、MAC 调度、HARQ、跨层策略等业务模块驻留运行

业务模块驻留运行

依赖：依赖 HAL 硬件能力、模块加载器加载部署，由调度器实时调度执行。

(9) 运行监控与自愈回退层

包含：运行监控与健康检查、异常判决器、安全回退控制器

定位：全链路实时监控、异常自动判定、故障自愈与降级回退

核心流程

1. 运行监控：模块心跳、通信性能、吞吐时延、资源占用、异常日志全量监控

2. 异常判决：依据 BLER/时延超限、模块崩溃、资源超限、协议/安全违规等条件触发异常

3. 安全回退控制：卸载异常模块、恢复默认配置、切回低速控制信道、维持最小信令、联动 AI 重生成优化方案

闭环：回退后重新联动控制信道与模块加载器，实现系统自稳。

(10) 外部互联接口层

包含三大外部对接接口：AI 通信编排器接口、数字孪生沙箱接口、上层业务/任务接口

1. AI 通信编排器接口

- 接收 AI 优化候选方案、下发验证后模块
- 上报运行状态与性能反馈、请求 AI 重规划

2. 数字孪生沙箱接口

- 上报实时状态数据、接收仿真验证结果
 - 加载仿真签名模块、支持回退策略预演测试
3. 上层业务/任务接口
- 承载应急通信、卫星通信、专用通信、低时延高可靠行业业务
- 定位：打通内核与智能算法、仿真测试、行业业务的双向交互通道。

4.AI 编程实现思路

如果想用 AI 完成这个原型系统“整套代码”的编写，不能简单理解为“让大模型一次性把所有代码生成出来”，而应采用 AI 辅助的方式：先由专家定义系统架构、接口规范、安全边界和测试标准，再让 AI 按照模块逐步生成代码、单元测试、仿真脚本、接口文档和部署脚本，最后通过自动化测试、数字孪生仿真、代码审查和实物验证不断迭代。

建议使用“AI 软件工程智能体团队”可以设置多个 AI 角色协同工作：1) 系统架构师，负责拆分模块，定义接口，设计目录结构，生成架构文档；2) 通信算法工程师 AI，负责 PHY 算法，MAC 策略，信道模型，链路仿真。3) 软件工程师 AI，负责，Python/C++代码；配置系统；4) 测试工程师 AI，5) 安全审查 AI，6) 文档工程师 AI。实际操作中，可以用 Claude Code、Cursor、GitHub Copilot、ChatGPT API、OpenAI Codex 类工具或自建多 Agent 框架实现。

先由专家写出系统架构和模块接口，不让 AI 自由设计底层架构；将系统拆成几十到数百个小模块，每个模块有明确规格：

1. 明确输入输出接口；
2. 明确数据结构；
3. 明确状态机；
4. 明确性能指标；
5. 明确测试用例；
6. 明确异常处理和回退逻辑。

然后让 AI 逐模块生成代码，并要求 AI 同时生成：源代码、单元测试、接口文档、示例配置、仿真脚本、日志和异常处理、CI/CD 测试脚本。

AI 可以承担这个项目中大量代码的初稿生成、测试生成、文档生成和脚本自动化工作，但必须建立在严格的架构设计、接口规范、自动化测试、数字孪生。AI 不是“自由发挥的程序员”，而是一个在严格软件工程规范下工作的“代码生成工厂”。不要让 AI 直接写“大系统”，而是让 AI 写“可验证的小模块”。

如果直接要求 AI “写一个 AI 自演进 RAN 系统”，生成的代码大概率不可运行、不可维护、不可验证。

第一阶段的目标，尝试用一个可演示、可复现、可演进的小系统，证明 AI 按需生成通信能力是可行的。